PREDICTIVE AUTO SCALING MINI PROJECT

A comparative study of reactive threshold-based and predictive auto scaling

Abstract

Cloud auto scaling automatically adjusts computing resources according to workload, but a purely reactive policy can respond late to sudden demand changes, repeatedly scale up and down during fluctuating demand, or maintain more capacity than necessary. This mini project proposes a predictive auto scaling controller that combines workload prediction, capacity estimation, a safety margin, multi-condition decisions, hysteresis, and cooldown periods. A baseline threshold-based auto scaler is compared with the proposed controller using a controlled workload simulation. The project evaluates resource usage, scaling actions, response-time behavior, SLA violations, and estimated compute cost. The implementation is designed to run locally using Python and can be mapped to a cloud deployment using Docker, Locust, Google Cloud Compute Engine Managed Instance Groups, Load Balancing, and Cloud Monitoring. The study is intended as a reproducible academic prototype: the numerical results are simulation results and should not be interpreted as production-cloud measurements.

Introduction

Cloud applications experience changing workloads throughout the day. A web service may receive only a small number of requests during normal periods but may receive a sudden burst during a sale, examination-result announcement, registration deadline, or viral event. If the application keeps a fixed number of servers, resources are wasted during low demand and performance can degrade during high demand.

Auto scaling addresses this problem by dynamically changing the number of running instances. Scale-out adds instances when demand increases, while scale-in removes instances when demand decreases. A common reactive approach uses a metric such as CPU utilization: if utilization crosses an upper threshold, more instances are added; if it remains below a lower threshold, instances are removed.

Although threshold-based scaling is simple, it has a fundamental limitation: the decision is based mainly on the current state, while provisioning takes time. By the time a scaling action completes, the workload may already have increased further. Fluctuating workloads can also cause oscillation, while conservative thresholds can increase cost. Therefore, this project investigates a predictive controller that estimates near-future demand and combines prediction with safeguards before changing capacity.

Problem Statement

The problem addressed in this project is the design of an auto scaling mechanism that can maintain application performance under changing workloads while avoiding unnecessary resource consumption.

The baseline problem is formulated as follows: given a changing request rate and a fixed per-instance processing capacity, decide when to add or remove instances using current utilization thresholds.

The proposed research problem is: can a lightweight predictive controller reduce delayed scaling, unnecessary scaling actions, and resource usage compared with a reactive threshold controller while maintaining acceptable response-time/SLA behavior?

The project specifically investigates six practical issues:

1. Scaling delay during sudden workload growth.

2. Scaling oscillation or thrashing when workload fluctuates around thresholds.

3. Decisions based on only one metric.

4. Over-provisioning during prolonged low demand.

5. Under-provisioning during sudden bursts.

6. The trade-off between performance protection and cloud resource cost.

Literature Review

Auto scaling research broadly contains reactive, predictive, and hybrid approaches.

Reactive auto scaling is based on observed metrics such as CPU utilization, request rate, queue length, or latency. It is easy to implement and has low computational overhead, but it reacts after the workload has already changed. This makes it vulnerable to sudden bursts and provisioning delays.

Predictive auto scaling estimates future workload from historical observations. Common forecasting families include moving averages, exponential smoothing, regression, time-series models, and machine-learning models. Prediction can provide advance notice of demand changes, but inaccurate forecasts can themselves cause incorrect scaling.

Hybrid approaches combine prediction with reactive safety checks. This is important because prediction is uncertain: a controller should not rely on a forecast alone when current utilization or latency indicates immediate overload.

Control techniques such as hysteresis and cooldown are also widely useful in dynamic resource management. Hysteresis uses different conditions for scaling out and scaling in, preventing repeated toggling near a single threshold. Cooldown prevents another scaling action immediately after a previous action, allowing the system to observe the effect of provisioning.

This project follows the hybrid idea and implements a lightweight predictor rather than a computationally expensive forecasting model. The objective is not to claim a universally superior algorithm, but to create an understandable, reproducible mini-project that demonstrates why prediction and control safeguards can improve auto scaling decisions under selected workload patterns.

Proposed Solution Architecture and Methodology

The proposed system contains five logical stages.

Stage 1 — Monitoring: The controller collects request rate, estimated utilization, active instances, and response-time information from the application/workload simulator.

Stage 2 — Prediction: A rolling history is used to estimate near-future request demand. The implementation uses a trend-adjusted moving-average predictor so that the project remains lightweight and easy to reproduce.

Stage 3 — Capacity planning: Required instances are estimated using the predicted request rate, the known per-instance capacity, and a safety margin.

Required instances = ceil(Predicted request rate × (1 + safety margin) / Per-instance capacity)

For example, if predicted demand is 900 requests/minute, one instance can process 220 requests/minute, and the safety margin is 20%, then:

Required instances = ceil(900 × 1.20 / 220) = ceil(4.91) = 5 instances.

Stage 4 — Decision control: The required capacity is not applied blindly. Hysteresis separates scale-out and scale-in conditions, and cooldown prevents repeated actions within a short interval. Additional consecutive-observation rules make scale-in more conservative than scale-out.

Stage 5 — Evaluation: The baseline and proposed controllers are exposed to the same workload trace. Their instance counts, scaling actions, response times, SLA violations, and estimated compute usage are recorded and compared.

The architecture diagram inserted in this report shows the complete data flow from users/workload through monitoring and prediction to the scaling controller and benchmarking layer.

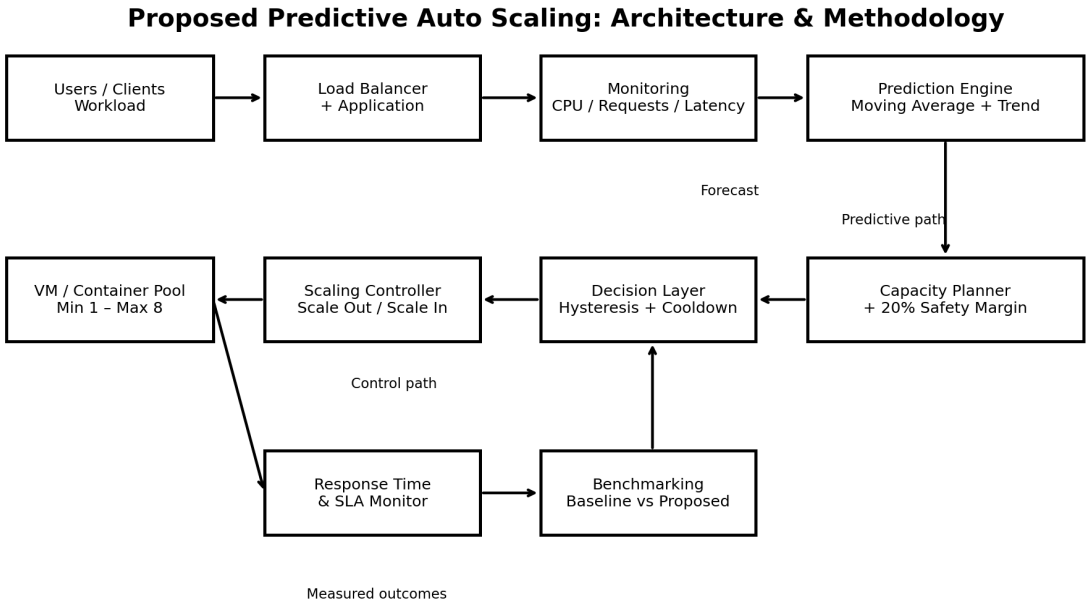


Figure: Proposed predictive auto scaling architecture and methodology.

Cases Solved by the Project

The project is designed to evaluate the controller against the following cases.

Case 1 — Stable workload: Demand remains relatively constant. The objective is to avoid unnecessary scaling actions.

Case 2 — Gradual increase: Workload increases slowly over time. The objective is to add capacity before utilization becomes persistently high.

Case 3 — Sudden spike: Request rate increases sharply. The objective is to reduce the delay associated with purely reactive scaling.

Case 4 — Sudden burst followed by recovery: Demand rises and then quickly falls. The objective is to prevent excessive scale-out and premature scale-in.

Case 5 — Fluctuating workload: Demand repeatedly moves around a threshold. The objective is to demonstrate the effect of hysteresis and cooldown in reducing oscillation.

Case 6 — Low-demand period: Demand remains low for a sustained period. The objective is to release unnecessary instances without repeatedly changing capacity.

Case 7 — Stress / capacity-limit scenario: Demand is deliberately pushed beyond comfortable capacity. This case is useful for making SLA-violation behavior visible. Results from such a scenario must be described as a controlled stress test, not as proof of production superiority.

Case 8 — Prediction error: The forecast is intentionally imperfect or workload changes unexpectedly. The objective is to verify that reactive safety conditions can protect the system when prediction is wrong.

Experimental Setup

The experimental prototype uses a synthetic workload trace containing stable periods, gradual variations, spikes, and bursts. The default simulation uses a per-instance capacity of 220 requests/minute, a minimum of 1 instance, and a maximum of 8 instances.

Baseline controller:

• Initial instances: 2

• Scale-out threshold: estimated utilization > 70%

• Scale-in threshold: estimated utilization < 30%

• Cooldown: 5 minutes

• Capacity bounds: 1–8 instances

Proposed controller:

• Initial instances: 2

• Predictor: rolling 12-point trend-adjusted moving average

• Safety margin: 20%

• Capacity bounds: 1–8 instances

• Scale-out protection: required capacity increase or high current load for consecutive observations

• Scale-in protection: lower predicted requirement plus sustained low current utilization

• Cooldown: 5 minutes

The simulator estimates response time using a queue/load-based relationship and marks an SLA violation when simulated response time exceeds 200 ms. This threshold is a project-level experimental assumption, not a universal cloud SLA.

The implementation uses Python, NumPy, Pandas, scikit-learn-compatible tooling where required, Matplotlib, Flask for a simple service layer, Docker for reproducibility, and Locust for optional load generation. The optional cloud mapping uses Google Cloud Compute Engine Managed Instance Groups, Load Balancing, and Cloud Monitoring.

Benchmarking Scope

The benchmark compares the baseline reactive controller and the proposed predictive controller under identical workload traces and configuration limits.

Primary metrics:

• Average number of active instances.

• Peak number of instances.

• Number of scale-out and scale-in actions.

• Total scaling actions.

• Average response time.

• Peak response time.

• Number and percentage of SLA violations.

• Estimated compute-resource cost units.

• Prediction error for the proposed controller.

The benchmark should be repeated for stable, gradual, spike, burst, fluctuating, low-demand, and stress workloads. For fair comparison, both controllers must receive the same workload sequence, instance capacity, minimum/maximum capacity, observation interval, and evaluation duration.

The current project results are simulation-based. They demonstrate behavior under the defined model; they do not establish that the proposed controller will outperform every cloud provider's native autoscaler or every workload in production.

Limitations

1. The workload is synthetic and may not represent every real application.

2. The current predictor is intentionally lightweight; it is not a full production-grade time-series forecasting system.

3. VM/container startup time is represented indirectly rather than measured on real cloud hardware.

4. The response-time model is simulated and therefore does not capture network latency, database bottlenecks, garbage collection, I/O, or application-specific processing.

5. The cost metric is an estimated compute-cost unit rather than a real cloud billing amount.

6. The project focuses primarily on horizontal scaling and does not model vertical scaling in detail.

7. Multi-service microservice dependencies are outside the present scope.

8. Prediction errors can still cause unnecessary or delayed scaling; safety rules reduce but do not eliminate this risk.

9. The maximum instance count and capacity values are experimental parameters and must be tuned for a real deployment.

Future Scope

The project can be extended in several directions.

1. Replace the moving-average predictor with LSTM, GRU, Temporal Convolutional Networks, XGBoost, or other forecasting models.

2. Use multi-metric forecasting based on request rate, CPU, memory, queue length, latency, and error rate.

3. Learn the capacity of each instance dynamically instead of assuming a fixed 220 requests/minute.

4. Include measured VM/container startup and shutdown times.

5. Implement reinforcement-learning-based scaling policies for adaptive decision making.

6. Extend the controller to Kubernetes Horizontal Pod Autoscaler or KEDA.

7. Deploy the complete controller on Google Cloud/AWS/Azure and compare it with native autoscaling.

8. Add workload traces from real web services or public cloud benchmark datasets.

9. Introduce cost-aware optimization using actual cloud pricing.

10. Add anomaly detection for unexpected workload spikes.

11. Evaluate energy consumption and carbon-aware scaling.

12. Extend the controller to multi-tier applications where web servers, application servers, and databases scale together.

13. Add confidence intervals to forecasts so that safety margins adapt to prediction uncertainty.

14. Run long-duration experiments and statistical significance tests over multiple workload seeds.

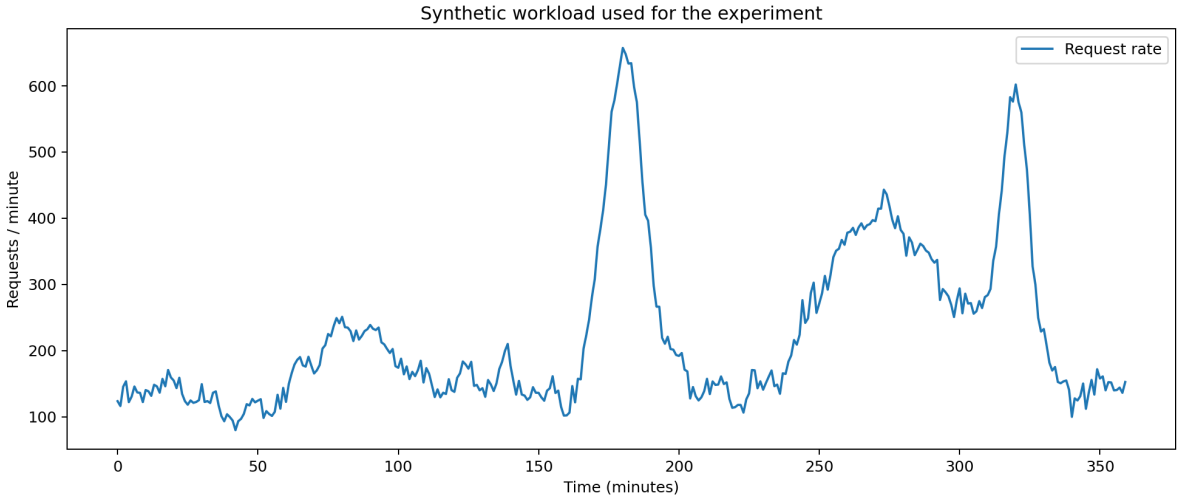
Conclusion

This mini project presents a practical predictive auto scaling framework that combines lightweight workload forecasting with capacity planning, safety margin, hysteresis, and cooldown control. The baseline provides a simple reactive reference, while the proposed controller attempts to anticipate demand and avoid unnecessary capacity changes. The project is structured so that the same workload can be replayed against both controllers, making benchmarking transparent and reproducible. The most important contribution is not a claim of universal superiority, but a complete experimental framework for studying the trade-off between application performance, scaling responsiveness, stability, and resource usage.

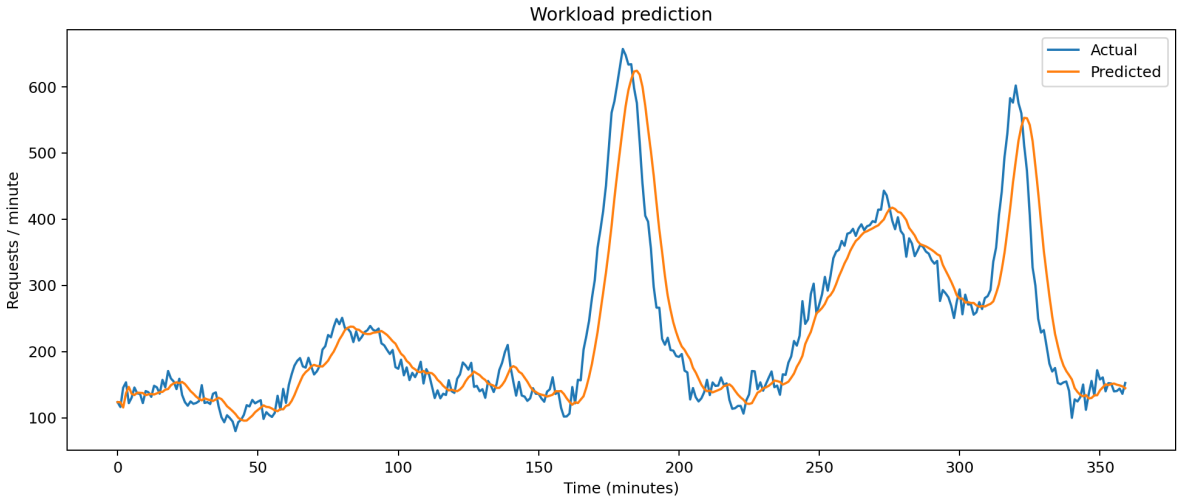
Existing Simulation Results

Unnamed: 0	average_instances	peak_instances	scale_actions	sla_violations	sla_rate_pct	estimated_compute_cost_units
Baseline	1.9333333333333331	4	21	0	0.0	6.96
Proposed	1.7944444444444445	4	17	0	0.0	6.460000000000001

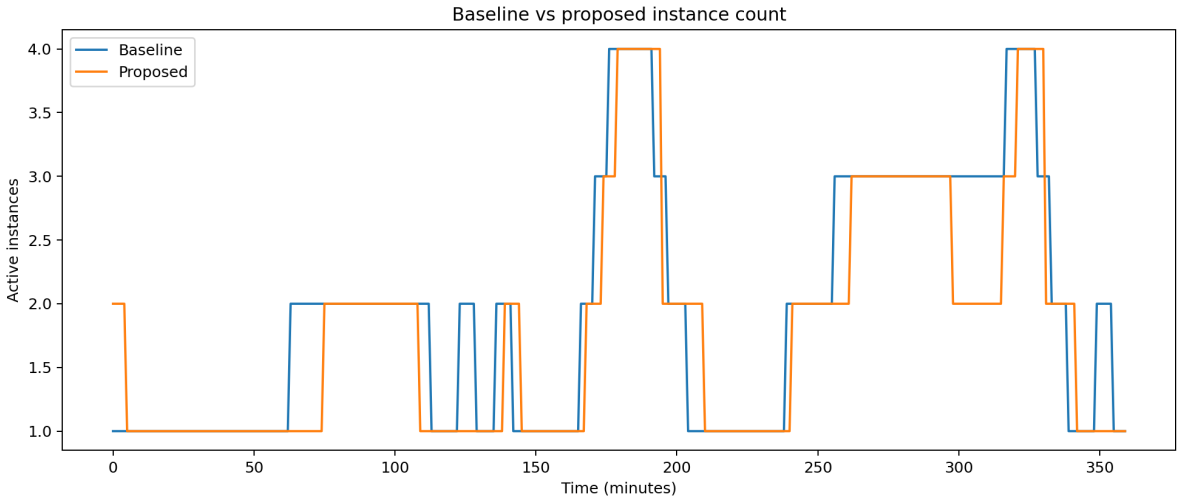
Supporting Experimental Figures



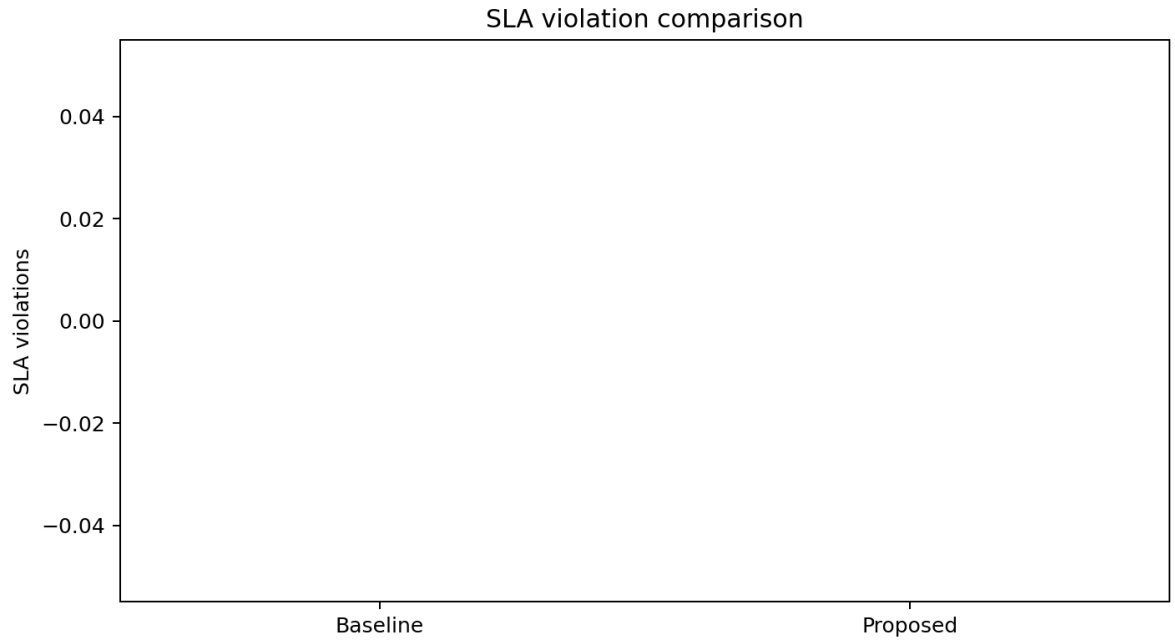
Workload



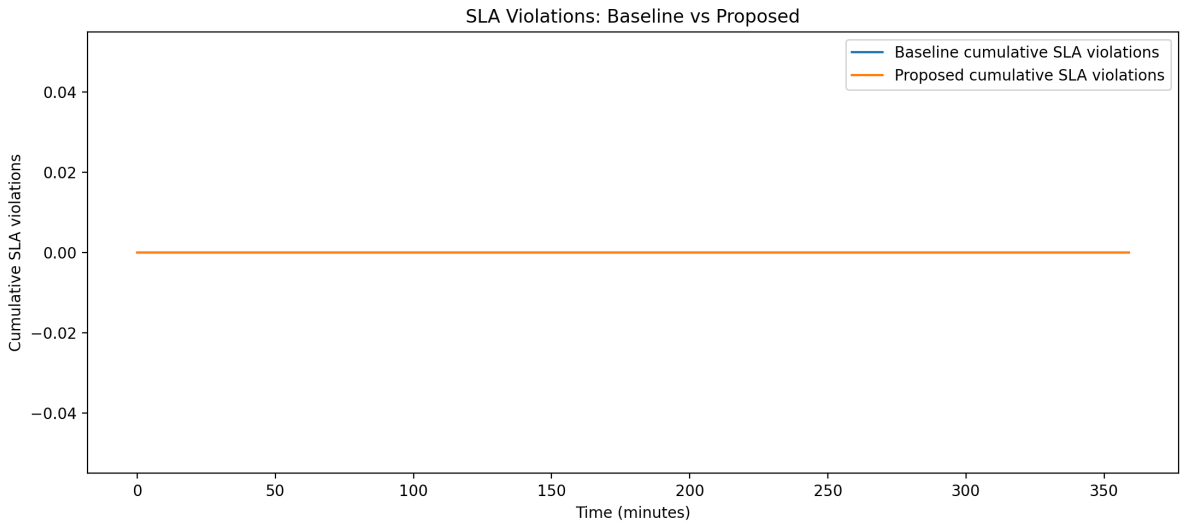
Prediction



Instances Comparison



Sla Comparison



Sla Violation Graph